

Got it — you want a **C# struct** that is exactly **6 bytes (48 bits)** in size, where **each bit has a named meaning** for status flags.

C# does not have native C-style bitfields, so we achieve this by:

- Using a **fixed-size byte array** inside an **unsafe struct** with `[StructLayout(LayoutKind.Sequential, Pack = 1)]` to ensure exact size.
- Providing **properties** that get/set individual bits using bitwise operations.
- Ensuring the struct is exactly **6 bytes**.

Complete Example

```
using System;
using System.Runtime.InteropServices;

[StructLayout(LayoutKind.Sequential, Pack = 1)]
public unsafe struct StatusFlags
{
    // Fixed-size buffer of exactly 6 bytes (48 bits)
    public fixed byte data[6];

    // Helper to get a bit at a given position (0-47)
    private bool GetBit(int bitIndex)
    {
        if (bitIndex < 0 || bitIndex >= 48)
            throw new ArgumentOutOfRangeException(nameof(bitIndex));

        int byteIndex = bitIndex / 8;
```

```
    int bitPos = bitIndex % 8;
    fixed (byte* ptr = data)
    {
        return (ptr[byteIndex] & (1 << bitPos)) != 0;
    }
}

// Helper to set a bit at a given position (0-47)
private void SetBit(int bitIndex, bool value)
{
    if (bitIndex < 0 || bitIndex >= 48)
        throw new ArgumentOutOfRangeException(nameof(bitIndex));

    int byteIndex = bitIndex / 8;
    int bitPos = bitIndex % 8;
    fixed (byte* ptr = data)
    {
        if (value)
            ptr[byteIndex] |= (byte)(1 << bitPos);
        else
            ptr[byteIndex] &= (byte)~(1 << bitPos);
    }
}

// Example named bits
public bool IsOnline
{
```

```
    get => GetBit(0);
    set => SetBit(0, value);
}

public bool HasError
{
    get => GetBit(1);
    set => SetBit(1, value);
}

public bool IsBusy
{
    get => GetBit(2);
    set => SetBit(2, value);
}

public bool NeedsMaintenance
{
    get => GetBit(3);
    set => SetBit(3, value);
}

// You can continue defining up to bit index 47...
}

class Program
{
```

```

static unsafe void Main()
{
    StatusFlags flags = new StatusFlags();

    flags.IsOnline = true;
    flags.HasError = false;
    flags.IsBusy = true;
    flags.NeedsMaintenance = true;

    Console.WriteLine($"IsOnline: {flags.IsOnline}");
    Console.WriteLine($"HasError: {flags.HasError}");
    Console.WriteLine($"IsBusy: {flags.IsBusy}");
    Console.WriteLine($"NeedsMaintenance: {flags.NeedsMaintenance}");

    Console.WriteLine($"Struct size: {sizeof(StatusFlags)} bytes");
}
}

```

Key Points

- `[StructLayout(LayoutKind.Sequential, Pack = 1)]` ensures no padding between fields.
- `fixed byte data[6]` creates exactly 6 bytes inside the struct.
- **Bit access** is done manually with `GetBit` and `SetBit`.
- **Unsafe code** is required for fixed buffers — compile with `/unsafe`.
- You can define **up to 48 named properties** for each bit.

✅ This approach is **memory-accurate** and **bit-precise**, making it suitable for **interop with hardware, network packets, or binary file formats**.

If you want, I can also give you a **safe (no unsafe keyword)** version using a `ulong` internally but still limited to 48 bits.

That would be easier to maintain but still exactly 6 bytes when serialized.

Do you want me to prepare that safe version too?